

LAMP: Linear Verification of Matrix Multiplication via Proximity Testing

Anonymous Submission

Abstract—Verifiable computation systems often need to prove large matrix multiplication statements, but a direct SNARK arithmetization of a $k \times k$ product requires $\mathcal{O}(k^3)$ constraints. Freivalds’ randomized check reduces the algebraic computation to vector-matrix products, but proving those products inside a SNARK still costs $\mathcal{O}(k^2)$ constraints.

We present **LAMP**, a matrix-multiplication checking protocol that combines Freivalds’ randomized check with proximity testing over linear error-correcting codes. The prover commits to encoded matrices and intermediate vectors before the sampled query positions are derived. The CP-SNARK circuit then checks only the sampled codeword positions and commits to the values used inside the circuit, while Merkle openings and CP-Link proofs ensure consistency between the in-circuit witnesses and the externally committed values. We prove soundness for this committed-input setting under the soundness of the SNARK backend, the binding of the commitments, the correctness of the CP-Link checks, and the distance of the code.

For a fixed number t of sampled positions, the main in-circuit SNARK relation has $\mathcal{O}(tk)$ constraints, with additional $\mathcal{O}(t \log n) + E_{\text{link}}(k)$ backend work for Merkle openings and CP-Link checks. We implement **LAMP** in Go and compare it with a Freivalds-based SNARK circuit. In the matrix benchmark at $k = 2^{12}$, **LAMP** reduces the constraint count by $30.3\times$ and shortens proof generation time by $8.34\times$; verification stays below 0.01 seconds in the measured range.

Index Terms—Verifiable computation, SNARKs, Proximity testing, Linear codes, Freivalds’ algorithm, Merkle trees, Commitment linking

1. Introduction

Large-scale matrix multiplication is a recurring bottleneck in verifiable computation. A prover may wish to convince a verifier that matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ satisfy $\mathbf{AB} = \mathbf{C}$, without requiring either the verifier or the SNARK circuit to recompute the entire product. This problem appears throughout verifiable inference and training for modern neural networks. In particular, projection and attention layers often involve matrix multiplications with dimension $k \geq 1,000$. A naïve arithmetization of a single $k \times k$ matrix multiplication inside a SNARK circuit generates $\mathcal{O}(k^3)$ arithmetic constraints, which is impractical even for moderate values of k .

The $\mathcal{O}(k^2)$ barrier. Freivalds’ classical randomized check [1] reduces this cost from $\mathcal{O}(k^3)$ to $\mathcal{O}(k^2)$. Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and a random vector $\mathbf{r}$, one verifies

$$\mathbf{rAB} = \mathbf{rC}$$

instead of recomputing the full product. Several recent systems, including Mystique [2], zkML [3], and the Freivalds-based baseline in [4], adopt this strategy by directly encoding the resulting vector-matrix products as SNARK constraints. The resulting $\mathcal{O}(k^2)$ -size circuit is an improvement, but it remains costly. In our optimized baseline, Freivalds already requires 50.5 million R1CS constraints and 405.04 s of proving time at $k = 4,096$. Thus, a significant gap remains between the $\mathcal{O}(k^2)$ SNARK cost and practical large-scale matrix verification.

Our insight: From computation to checking. We observe that the $\mathcal{O}(k^2)$ barrier is not inherent to the verification problem itself. Rather, it arises because existing approaches *compute* vector-matrix products inside the SNARK circuit. Our key idea is to move from “computing inside the circuit” to “checking inside the circuit.” The prover performs the expensive matrix arithmetic outside the SNARK and commits to the relevant intermediate values. The verifier then checks only a small number of lightweight algebraic relations that are sufficient to detect inconsistencies.

The technical core of this approach is *proximity testing over linear error-correcting codes*. By encoding each matrix row-wise using a systematic linear code $\mathcal{C}$ with relative distance δ , the vector-matrix products

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}$$

can be checked column by column in the encoded domain. The linearity of $\mathcal{C}$ ensures that a single inner product, or “fold,” over one codeword column is sufficient to check one coordinate of the encoded product. If the prover cheats in any of these relations, the corresponding codewords disagree on at least a δ -fraction of positions, and a random query detects the inconsistency with probability at least δ .

The key point is that the SNARK circuit does not take the full matrices as witnesses. Instead, it receives only t sampled columns and checks local algebraic relations on those columns, including fold computations and the codeword encoding consistency of the intermediate vectors $\mathbf{x}, \mathbf{y}$, and $\mathbf{z}$. We use a commit-and-prove SNARK to commit to the sampled block witnesses used inside the circuit. Together with the SNARK proof, the prover sends Merkle membership proofs for the queried leaves under rt_{ABC} and rt_{XYZ} , as well as CP-Link proofs showing that the SNARK-side sampled blocks and the opened Pedersen leaves contain

the same data. By verifying these proofs, the verifier is assured that the values used inside the circuit are consistent with the committed data fixed before the query positions were derived. As a result, the in-circuit checking cost is $\mathcal{O}(tk)$, while the additional opening and linking work is $\mathcal{O}(t \log n) + E_{\text{link}}(k)$. Since t is the number of sampled codeword positions and is independent of the matrix dimension k , the protocol has linear in-circuit cost in the matrix dimension.

Contributions. We present LAMP, a protocol for efficiently proving matrix multiplication statements. Our contributions are as follows.

- **A code-based protocol for matrix multiplication.** We propose LAMP, which combines Freivalds’ randomized reduction with proximity testing over linear error-correcting codes. Instead of computing vector-matrix products inside the SNARK circuit, LAMP encodes the matrices and intermediate vectors as codewords and checks only randomly sampled codeword positions. By exploiting the linearity of the code, each sampled position can be verified through local fold relations, reducing the in-circuit checking cost from $\mathcal{O}(k^2)$ to $\mathcal{O}(tk)$, which is linear in the matrix dimension for fixed t .
- **Soundness analysis.** We prove the soundness of LAMP. In particular, we show that if $\mathbf{AB} \neq \mathbf{C}$, then any prover that commits to inconsistent encoded values is detected with probability determined by the code distance and the number of sampled positions. The analysis also accounts for the Merkle openings and CP-Link proofs that bind the values checked inside the SNARK circuit to the data committed before proof generation.
- **Implementation and evaluation.** We implement LAMP in Go and evaluate it against a Freivalds-based SNARK circuit for matrix dimensions from $k = 2^7$ to 2^{13} . At $k = 2^{12}$, LAMP reduces the constraint count by $30.3\times$ and shortens proof generation time by $8.34\times$, while verification remains independent of the matrix dimension and takes less than 0.01 s. We also evaluate LAMP on a GPT-2 workload with sequence length 2^{10} , where it reduces the constraint count by $2.3\times$ and proving time by $1.78\times$, showing that the approach extends to matrix-heavy neural network workloads.

2. Preliminaries

We introduce notation and recall the building blocks used in our construction. Throughout, $\mathbb{F}$ denotes a finite field of size $|\mathbb{F}| = p$ for a prime p , and λ denotes the security parameter. We write $\text{negl}(\lambda)$ for any function that is negligible in λ , and PPT for probabilistic polynomial-time.

2.1. Notation

Matrices are denoted by bold uppercase letters $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$, and vectors by bold lowercase letters

$\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}^k$. For a vector $\mathbf{x}$, we write $\mathbf{x}[i]$ for its i -th entry. For a matrix $\mathbf{M}$, we write $\mathbf{M}[i, j]$ for the entry in the i -th row and j -th column. We denote the i -th row of $\mathbf{M}$ by $\mathbf{M}[i, *]$, and we will often write it simply as $\mathbf{M}[i]$. Similarly, we denote the j -th column of $\mathbf{M}$ by $\mathbf{M}[* , j]$. We write $[n] = \{0, 1, \dots, n-1\}$. For $t \in \mathbb{N}$, $I \leftarrow [n]^t$ denotes the uniform sampling of t indices from $[n]$. Similarly, $r \leftarrow \mathbb{F}$ denotes uniform random sampling from $\mathbb{F}$.

2.2. Linear Error-Correcting Codes

A linear code over $\mathbb{F}$ is an $\mathbb{F}$ -linear map $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ defined by a generator matrix $\mathbf{G} \in \mathbb{F}^{k \times n}$, so that $\mathcal{C}(\mathbf{m}) = \mathbf{m} \cdot \mathbf{G}$ for any input vector $\mathbf{m} \in \mathbb{F}^k$. The code has *rate* $\rho = k/n$ and *minimum distance* $d = \min_{\mathbf{c} \neq \mathbf{c}'} \Delta(\mathbf{c}, \mathbf{c}')$, where the minimum is taken over distinct codewords $\mathbf{c}, \mathbf{c}' \in \mathcal{C}$, and Δ denotes the Hamming distance. The *relative distance* is $\delta = d/n$. A code is *systematic* if the first k coordinates of $\mathcal{C}(\mathbf{m})$ equal $\mathbf{m}$ itself.

We write $\text{Enc}(\mathbf{m}) := \mathcal{C}(\mathbf{m})$ for the encoding of an input vector. For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, we define the *row-wise encoding* $\widehat{\mathbf{M}} := \text{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$ as the matrix whose i -th row is $\text{Enc}(\mathbf{M}[i, *])$ for each $i \in [k]$. The j -th column of $\widehat{\mathbf{M}}$ is denoted by $\widehat{\mathbf{M}}[* , j] \in \mathbb{F}^k$ for each $j \in [n]$.

The key property we exploit is *linearity*: for any coefficients $r_1, \dots, r_k \in \mathbb{F}$,

$$\sum_{i=1}^k r_i \cdot \widehat{\mathbf{M}}[i, *] = \text{Enc}\left(\sum_{i=1}^k r_i \cdot \mathbf{M}[i, *]\right). \quad (1)$$

Definition 2.1 (Fold). For vectors $\mathbf{c}, \mathbf{r} \in \mathbb{F}^k$ with $\mathbf{c} = (c_0, \dots, c_{k-1})^\top$ and $\mathbf{r} = (r_0, \dots, r_{k-1})$, define

$$\text{Fold}(\mathbf{r}, \mathbf{c}) := \langle \mathbf{r}, \mathbf{c} \rangle = \sum_{i=0}^{k-1} r_i \cdot c_i.$$

Equivalently, for the j -th column of a row-wise encoded matrix, $\text{Fold}(\mathbf{r}, \widehat{\mathbf{M}}[* , j])$ equals the j -th coordinate of $\text{Enc}(\mathbf{rM})$. This follows directly from the linearity of the code (Equation 1).

Concrete instantiations. Our protocol is compatible with any linear code. Two natural choices are: (i) *Reed–Solomon codes*, where $\mathcal{C}(\mathbf{m})$ evaluates the polynomial $\sum_i m_i X^i$ over a domain of size $n > k$, achieving the optimal (MDS) distance $\delta = 1 - k/n + 1/n$; and (ii) *expander-based codes* as used in Brakedown [5], which achieve linear-time encoding $\mathcal{O}(n)$ with constant relative distance. We discuss the trade-offs in Section 4.6.

2.3. Commitment Schemes

A commitment scheme allows a committer to bind itself to a message and later open it. Some commitment schemes also hide the message before opening; our soundness arguments use only binding. In this work, we use two types of commitments: Merkle tree commitments and Pedersen vector commitments.

2.3.1. Merkle tree commitments. A Merkle tree commitment over a collision-resistant hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a vector commitment for a vector $\mathbf{v} = (\mathbf{v}[1], \dots, \mathbf{v}[n]) \in \mathbb{F}^n$. It consists of the following algorithms.

- $\text{rt} \leftarrow \text{MT.Com}(\mathbf{v}; \mathbf{o})$: outputs a root hash rt obtained by constructing a binary hash tree with randomized leaves $\ell_j = H(\mathbf{v}[j], \mathbf{o}[j])$ for $j \in [n]$. Internal nodes are computed by hashing their two children.
- $(\mathbf{v}[j], \mathbf{o}_j, \pi_j) \leftarrow \text{MT.Open}(\mathbf{v}, \mathbf{o}, j)$: outputs the leaf value $\mathbf{v}[j]$, its opening randomness $\mathbf{o}_j$, and its authentication path π_j for position $j \in [n]$.
- $b \leftarrow \text{MT.Verify}(\text{rt}, j, \mathbf{v}[j], \mathbf{o}[j], \pi_j)$: recomputes the randomized leaf $\ell_j = H(\mathbf{v}[j], \mathbf{o}[j])$ and outputs $b = 1$ if the authentication path is consistent with root rt at position j , and outputs $b = 0$ otherwise.

Merkle tree commitments are *position-binding*: under the collision resistance of H , no PPT adversary can produce a root rt , an index $j \in [n]$, two distinct openings $(v, o) \neq (v', o')$, and authentication paths π_j, π'_j such that

$$\text{MT.Verify}(\text{rt}, j, v, o, \pi_j) = \text{MT.Verify}(\text{rt}, j, v', o', \pi'_j) = 1,$$

except with probability $\epsilon_{\text{merkle}}(\lambda) = \text{negl}(\lambda)$.

2.3.2. Pedersen commitments. Pedersen commitments can be viewed as algebraic vector commitments over a group $\mathbb{G}$ of prime order q . Let $\text{ck} = (G_1, \dots, G_n, H)$ be a commitment key consisting of fixed generators in $\mathbb{G}$. A Pedersen commitment is defined by the following algorithms.

- $\text{cm} \leftarrow \text{Ped.Com}(\text{ck}, \mathbf{v}; \mathbf{o})$: outputs a commitment $\text{cm} := \sum_{i=1}^n \mathbf{v}[i]G_i + \mathbf{o}H \in \mathbb{G}$ for a vector $\mathbf{v} \in \mathbb{F}^n$ and randomness $\mathbf{o} \leftarrow \mathbb{F}$.
- $\pi \leftarrow \text{Ped.Open}(\mathbf{v}, \mathbf{o})$: outputs the opening $\pi = (\mathbf{v}, \mathbf{o})$ consisting of the committed vector and the randomness.
- $b \leftarrow \text{Ped.Verify}(\text{ck}, \text{cm}, \mathbf{v}, \mathbf{o})$: outputs $b = 1$ if $\text{cm} \stackrel{?}{=} \sum_{i=1}^n \mathbf{v}[i]G_i + \mathbf{o}H$, and outputs $b = 0$ otherwise.

Pedersen commitments are *perfectly hiding* and *computationally binding*: under the discrete logarithm assumption, no PPT adversary can produce a commitment cm , two distinct vectors $\mathbf{v} \neq \mathbf{v}'$, and randomness values $\mathbf{o}, \mathbf{o}'$ such that

$$\text{Ped.Verify}(\text{ck}, \text{cm}, \mathbf{v}, \mathbf{o}) = \text{Ped.Verify}(\text{ck}, \text{cm}, \mathbf{v}', \mathbf{o}') = 1,$$

except with probability $\epsilon_{\text{ped}}(\lambda) = \text{negl}(\lambda)$.

They are also *additively homomorphic*: for $\text{cm}_1 = \text{Ped.Com}(\text{ck}, \mathbf{v}_1; \mathbf{o}_1)$ and $\text{cm}_2 = \text{Ped.Com}(\text{ck}, \mathbf{v}_2; \mathbf{o}_2)$, $\text{cm}_1 + \text{cm}_2 = \text{Ped.Com}(\text{ck}, \mathbf{v}_1 + \mathbf{v}_2; \mathbf{o}_1 + \mathbf{o}_2)$.

2.4. Succinct Non-interactive Arguments of Knowledge

A SNARK (succinct non-interactive argument of knowledge) for an NP relation $\mathcal{R}$ is a tuple of algorithms $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$.

- $\text{pp} \leftarrow \Pi.\text{Setup}(1^\lambda, \mathcal{R})$: generates public parameters pp for the relation $\mathcal{R}$.

- $\pi \leftarrow \Pi.\text{Prove}(\text{pp}, \mathbf{x}, \mathbf{w})$: outputs a proof π for a statement $\mathbf{x}$ and witness $\mathbf{w}$ satisfying $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$.
- $b \leftarrow \Pi.\text{Verify}(\text{pp}, \mathbf{x}, \pi)$: outputs $b = 1$ if π is accepted as a valid proof for $\mathbf{x}$, and outputs $b = 0$ otherwise.

Remark 1 (Security Properties). A SNARK scheme Π satisfies completeness, if an honestly generated proof for any valid statement-witness pair $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ is accepted by the verifier with overwhelming probability. It satisfies knowledge soundness if any PPT prover that produces an accepting proof for a statement $\mathbf{x}$ must know a valid witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, except with negligible probability. This is formalized by the existence of an efficient knowledge extractor. A SNARK may additionally satisfy zero knowledge, meaning that the proof reveals no information beyond the truth of the statement. This is modeled by a simulator Sim that can generate a proof indistinguishable from a real proof without knowing a valid witness. A SNARK satisfying zero knowledge is called a zk-SNARK.

2.4.1. Commit-and-Prove SNARKs. A commit-and-prove SNARK (CP-SNARK) [6], [7] is a variant of a SNARK that proves statements about witnesses bound to an external commitment. We call such a commitment a *witness commitment*. Let Com be a commitment scheme with commitment key ck . For an NP relation $\mathcal{R}$, a CP-SNARK proves knowledge of a witness $\mathbf{w} := (\mathbf{u}, \omega, \mathbf{o})$ such that $\widetilde{\text{cm}} = \text{Com}(\text{ck}, \mathbf{u}; \mathbf{o})$ and $(\mathbf{x}, \mathbf{u}, \omega) \in \mathcal{R}$. Here, $\mathbf{u}$ is the committed witness, $\mathbf{o}$ is the opening randomness of the witness commitment $\widetilde{\text{cm}}$, and ω is the remaining private witness. A CP-SNARK scheme is a tuple of algorithms $\Pi_{\text{cp}} = (\text{Setup}, \text{Prove}, \text{Verify})$.

- $\text{pp} \leftarrow \Pi_{\text{cp}}.\text{Setup}(1^\lambda, \mathcal{R})$: generates public parameters $\text{pp} = (\text{pk}, \text{vk}, \text{ck})$ for the relation $\mathcal{R}$, where pk is the proving key, vk is the verifying key, and ck is the commitment key.
- $(\pi_{\text{cp}}, \widetilde{\text{cm}}) \leftarrow \Pi_{\text{cp}}.\text{Prove}(\text{pp}, \mathbf{x}, \mathbf{u}, \omega, \mathbf{o})$: outputs a proof π_{cp} and a witness commitment $\widetilde{\text{cm}}$ such that $\widetilde{\text{cm}} = \text{Com}(\text{ck}, \mathbf{u}; \mathbf{o})$.
- $b \leftarrow \Pi_{\text{cp}}.\text{Verify}(\text{pp}, \mathbf{x}, \widetilde{\text{cm}}, \pi_{\text{cp}})$: outputs $b = 1$ if π_{cp} is accepted as a valid proof for $\mathbf{x}$ with respect to the witness commitment $\widetilde{\text{cm}}$, and outputs $b = 0$ otherwise.

2.5. CP-Link

A CP-Link proof is a proof system for linking commitments. It proves that two or more commitments are commitments to the same message, without revealing the message or the opening randomness.

Let Com be a commitment scheme. For commitment keys $\text{ck}_1, \dots, \text{ck}_t$ and commitments $\text{cm}_1, \dots, \text{cm}_t$, the linking relation is defined as

$$\mathcal{R}_{\text{link}} = \left\{ \left(((\text{ck}_i, \text{cm}_i)_{i=1}^t), (m, (\mathbf{o}_i)_{i=1}^t) \right) : \right. \\ \left. \text{cm}_i = \text{Com}(\text{ck}_i, m; \mathbf{o}_i) \text{ for all } i \in [t] \right\}.$$

A proof for $\mathcal{R}_{\text{link}}$ convinces the verifier that all commitments $\text{cm}_1, \dots, \text{cm}_t$ are commitments to the same message m .

A CP-Link proof system is a tuple of algorithms $\Pi_{\text{link}} = (\text{Setup}, \text{Prove}, \text{Verify})$.

- $\text{pp}_{\text{link}} \leftarrow \Pi_{\text{link}}.\text{Setup}(1^\lambda, \mathcal{R}_{\text{link}})$: generates public parameters for the linking relation.
- $\pi_{\text{link}} \leftarrow \Pi_{\text{link}}.\text{Prove}(\text{pp}_{\text{link}}, (\text{ck}_i, \text{cm}_i)_{i=1}^t, m, (o_i)_{i=1}^t)$: outputs a proof that the commitments open to the same message m .
- $b \leftarrow \Pi_{\text{link}}.\text{Verify}(\text{pp}_{\text{link}}, (\text{ck}_i, \text{cm}_i)_{i=1}^t, \pi_{\text{link}})$: outputs $b = 1$ if the proof is accepted, and outputs $b = 0$ otherwise.

2.6. Freivalds' Algorithm

Freivalds' algorithm [1] is a classical randomized algorithm for verifying matrix multiplication. Given $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$, it samples $\mathbf{r} \leftarrow \mathbb{F}^k$ and checks whether $\mathbf{r}(\mathbf{AB}) = \mathbf{rC}$. The correctness relies on the following:

Lemma 2.2 (Schwartz–Zippel [8], [9]). *Let $P(X_1, \dots, X_m)$ be a non-zero polynomial of total degree d over a finite field $\mathbb{F}$. Then for $r_1, \dots, r_m \leftarrow \mathbb{F}$ chosen uniformly and independently,*

$$\Pr[P(r_1, \dots, r_m) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

In our protocol, we use a structured variant: instead of sampling $\mathbf{r} \leftarrow \mathbb{F}^k$ uniformly, we sample one scalar $r \leftarrow \mathbb{F}$ and set $\mathbf{r} = (1, r, r^2, \dots, r^{k-1})$. This keeps the Freivalds challenge compact in the transcript and in the SNARK statement: the verifier records one field element rather than k independent ones. If $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, then for any non-zero column $\mathbf{d}_j$ of $\mathbf{D}$, the value $\langle \mathbf{r}, \mathbf{d}_j \rangle$ is a non-zero univariate polynomial in r of degree at most $k-1$. By Lemma 2.2, it vanishes with probability at most $(k-1)/|\mathbb{F}|$.

3. Overview of LAMP

This section gives a technical overview of LAMP before the formal construction in Section 4. The main message is that LAMP does not ask the SNARK circuit to compute a matrix product. Instead, the prover commits to encoded data, and the circuit checks only a small number of local consistency equations at randomly sampled codeword positions.

3.1. Problem Statement and Relation Reduction Chain

The starting point is the matrix multiplication relation

$$\mathbf{AB} = \mathbf{C}, \quad \mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}.$$

A direct SNARK arithmetization of this relation computes every entry of $\mathbf{AB}$. This requires k^3 scalar multiplications and therefore $\mathcal{O}(k^3)$ arithmetic constraints. This is the naïve relation:

$$\mathcal{R}_{\text{naive}} = \{(\mathbf{A}, \mathbf{B}, \mathbf{C}) : \mathbf{AB} = \mathbf{C}\}.$$

Freivalds' algorithm reduces the cubic check to a vector-matrix check. The classical algorithm samples a challenge vector $\mathbf{r} \in \mathbb{F}^k$ and checks

$$\mathbf{rAB} = \mathbf{rC}.$$

In LAMP, the verifier derives one scalar r and uses the structured vector $(1, r, \dots, r^{k-1})$, which keeps the public challenge compact. Equivalently, the prover introduces intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC},$$

and the verifier checks $\mathbf{y} = \mathbf{z}$. This gives the Freivalds relation

$$\mathcal{R}_{\text{Fr}} = \left\{ (\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{x}, \mathbf{y}, \mathbf{z}) : \begin{aligned} \mathbf{x} &= \mathbf{rA}, & \mathbf{y} &= \mathbf{xB}, & \mathbf{z} &= \mathbf{rC}, & \mathbf{y} &= \mathbf{z} \end{aligned} \right\}.$$

This removes the cubic matrix-matrix computation, but a SNARK circuit that computes $\mathbf{rA}$, $\mathbf{xB}$, and $\mathbf{rC}$ directly still performs $\mathcal{O}(k^2)$ scalar multiplications.

LAMP applies one more reduction. Rather than computing the full vector-matrix products inside the circuit, it checks the products only at randomly sampled positions in an encoded domain. The prover performs the expensive native computation outside the SNARK, commits to the encoded matrices and intermediate vectors, and the SNARK checks local equations of the form

$$\begin{aligned} \text{Enc}(\mathbf{x})[j] &= \text{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*], j), \\ \text{Enc}(\mathbf{y})[j] &= \text{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*], j), \\ \text{Enc}(\mathbf{z})[j] &= \text{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*], j). \end{aligned}$$

Only the queried columns $\widehat{\mathbf{A}}[*], j$, $\widehat{\mathbf{B}}[*], j$, $\widehat{\mathbf{C}}[*], j$ are used by the circuit. Thus, for t queried positions, the fold part of the circuit has cost $\mathcal{O}(tk)$, which is linear in k for fixed t .

3.2. Encoding and Proximity Testing

Let $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a systematic linear code with relative distance δ . For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, the prover row-wise encodes $\mathbf{M}$ as $\widehat{\mathbf{M}} = \text{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$.

The key identity is the linearity of the code. For any vector $\mathbf{u} \in \mathbb{F}^k$,

$$(\text{Fold}(\mathbf{u}, \widehat{\mathbf{M}}[*], 1), \dots, \text{Fold}(\mathbf{u}, \widehat{\mathbf{M}}[*], n)) = \text{Enc}(\mathbf{uM}).$$

Therefore, one coordinate of the encoded vector-matrix product can be checked using one encoded column. For example, if $\mathbf{x} = \mathbf{rA}$, then for every $j \in [n]$,

$$\widehat{\mathbf{x}}[j] = \text{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*], j).$$

Similarly, if $\mathbf{y} = \mathbf{xB}$ and $\mathbf{z} = \mathbf{rC}$, then

$$\widehat{\mathbf{y}}[j] = \text{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*], j), \quad \widehat{\mathbf{z}}[j] = \text{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*], j).$$

The prover commits to the encoded objects before the query positions are sampled. After the query set $I =$

$(j_1, \dots, j_t)$ is derived, the SNARK checks the above equations only for $j \in I$, together with the equality

$$\hat{\mathbf{y}}[j] = \hat{\mathbf{z}}[j].$$

This is a proximity test: if a claimed vector relation is false, then the two corresponding codewords are distinct. Since the code has relative distance δ , distinct codewords disagree on at least a δ -fraction of positions. Random sampling therefore detects an inconsistent relation with probability governed by δ and the number of sampled positions.

3.3. Commitments and CP-Link

The sampled checks above are meaningful only if the values used inside the SNARK are bound to data fixed before the relevant challenges are sampled. LAMP achieves this using two layers of commitments.

First, the prover commits to the encoded matrix columns. In our construction, the three encoded columns at position j are committed together:

$$\begin{aligned} \mathbf{m}_{ABC,j} &:= (\hat{\mathbf{A}}[* , j] \parallel \hat{\mathbf{B}}[* , j] \parallel \hat{\mathbf{C}}[* , j]), \\ \text{cm}_{ABC,j} &\leftarrow \text{Ped.Com}(\text{ck}_{ABC}, \mathbf{m}_{ABC,j}; o_{ABC,j}). \end{aligned}$$

The vector of Pedersen commitments is then committed with a Merkle tree:

$$\text{rt}_{ABC} \leftarrow \text{MT.Com}(\text{cm}_{ABC,1}, \dots, \text{cm}_{ABC,n}; \perp).$$

This root is fixed before the Freivalds challenge is derived.

After the Freivalds challenge is fixed, the prover computes the intermediate vectors and commits to their encoded coordinates. Conceptually, this includes the encoded symbols of $\hat{\mathbf{x}}$, $\hat{\mathbf{y}}$, and $\hat{\mathbf{z}}$. At a high level, the second Merkle root binds the encoded vector symbols used by the sampled checks:

$$\text{rt}_{XYZ} \leftarrow \text{MT.Com}(\text{cm}_{XYZ,1}, \dots, \text{cm}_{XYZ,n}; \perp).$$

The SNARK circuit does not verify Pedersen openings or Merkle paths internally. Instead, it uses a commit-and-prove SNARK to commit to the sampled witness values used by the circuit. The prover then supplies:

- Merkle openings showing that the external Pedersen commitments $\text{cm}_{ABC,j}$ and $\text{cm}_{XYZ,j}$ are authenticated under rt_{ABC} and rt_{XYZ} ;
- CP-Link proofs showing that the witness commitments from CP-SNARK and the external Pedersen commitments open to the same messages.

Thus the verifier obtains a chain of consistency:

$$\begin{array}{c} \text{SNARK witness values} \\ \Updownarrow \text{CP-Link} \\ \text{External Pedersen leaf commitments} \\ \Updownarrow \text{Merkle openings} \\ \text{pre-query Merkle roots.} \end{array}$$

This separation is essential for efficiency. The SNARK checks only arithmetic relations on field elements, while Merkle authentication and commitment linking are verified outside the circuit.

3.4. Soundness Intuition

The soundness argument has three layers.

Freivalds soundness. If $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{D} = \mathbf{AB} - \mathbf{C}$ is nonzero. With the structured challenge $\mathbf{r} = (1, r, \dots, r^{k-1})$, the vector $\mathbf{rD}$ is nonzero except with probability at most $(k-1)/|\mathbb{F}|$, by Schwartz–Zippel. Therefore, except with this probability, at least one of the vector relations in the Freivalds reduction is false.

Proximity soundness. Condition on the commitments being binding and fixed before the sampled positions are derived. If a vector relation is false, then the two associated encoded vectors are distinct codewords. Since the code has relative distance δ , they differ on at least δn positions. A random query misses this disagreement with probability at most $1 - \delta$, and t independent queries miss it with probability at most

$$(1 - \delta)^t.$$

Union-bounding over the local relations checked by the protocol gives the proximity term used in the security analysis.

Commitment and linking soundness. The distance argument applies only to values fixed before the query positions are sampled. Merkle position binding prevents the prover from opening a root to two different leaves at the same position. Pedersen binding prevents a committed leaf from being opened to two different messages. CP-Link ensures that the values used by the SNARK are the same values authenticated by the external commitments. Therefore, except for the binding and linking failure probabilities, any accepting proof corresponds to fixed encoded objects that pass the sampled proximity checks.

Putting the layers together, a cheating prover must either break one of the commitment/linking mechanisms, violate SNARK soundness, hit the Freivalds failure event, or avoid all sampled disagreements in the encoded domain. The formal theorem in Section 5 makes this union bound explicit.

4. The LAMP Protocol

This section gives the formal construction of LAMP, following the overview in Section 3. The protocol proves a matrix multiplication statement over committed encoded data. The SNARK circuit checks only sampled codeword positions, while Merkle openings and CP-Link proofs bind the sampled circuit witnesses to values fixed before the query positions are derived.

4.1. Committed Statement

Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$, and let $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a systematic linear code with relative distance δ . The target relation is

$$\mathcal{R}_{\text{MM}} = \{(\mathbf{A}, \mathbf{B}, \mathbf{C}) : \mathbf{AB} = \mathbf{C}\}.$$

The verifier does not receive the raw matrices as public input. Instead, the prover commits to row-wise encodings of

Symbol	Description
k, n, t	Matrix dimension, codeword length, and number of queried positions.
$\mathcal{C}$	Systematic linear code $\mathbb{F}^k \rightarrow \mathbb{F}^n$.
$\widehat{\mathbf{M}}$	Row-wise encoding of matrix $\mathbf{M}$.
$\widehat{\mathbf{v}}$	Encoding of vector $\mathbf{v}$.
rt_{ABC}	Merkle root over packed Pedersen commitments to encoded columns of $\mathbf{A}, \mathbf{B}, \mathbf{C}$.
rt_{XYZ}	Merkle root over packed Pedersen commitments to encoded symbols of $\mathbf{x}, \mathbf{y}, \mathbf{z}$.
$I = (j_1, \dots, j_t)$	Transcript-derived tuple of queried codeword positions.
$\widetilde{\text{cm}}_{ABC}, \widetilde{\text{cm}}_{XYZ}$	SNARK-side witness commitments to sampled packed values.
π_{mt}	Merkle authentication paths for queried leaves.
π_{link}	CP-Link proofs connecting SNARK-side commitments to external Pedersen commitments.

TABLE I. PROTOCOL-SPECIFIC NOTATION USED IN SECTION 4.

the matrices and proves that the committed objects satisfy the multiplication relation.

As in Section 2.6, LAMP derives one scalar $r \in \mathbb{F}$ and uses the structured Freivalds vector

$$\mathbf{r} = (1, r, r^2, \dots, r^{k-1}).$$

The prover then defines

$$\mathbf{x} = \mathbf{r}\mathbf{A}, \quad \mathbf{y} = \mathbf{x}\mathbf{B}, \quad \mathbf{z} = \mathbf{r}\mathbf{C}.$$

If $\mathbf{AB} = \mathbf{C}$, then $\mathbf{y} = \mathbf{z}$. Therefore, the matrix multiplication statement can be checked by verifying the three vector-product relations above and the final equality $\mathbf{y} = \mathbf{z}$.

4.2. Matrix Encoding and Commitment

The prover first encodes the input and output matrices row-wise:

$$\widehat{\mathbf{A}} = \text{Enc}(\mathbf{A}), \quad \widehat{\mathbf{B}} = \text{Enc}(\mathbf{B}), \quad \widehat{\mathbf{C}} = \text{Enc}(\mathbf{C}).$$

For each codeword position $j \in [n]$, it packs the three encoded columns into one vector

$$\mathbf{m}_{ABC,j} := (\widehat{\mathbf{A}}[* , j] \parallel \widehat{\mathbf{B}}[* , j] \parallel \widehat{\mathbf{C}}[* , j]) \in \mathbb{F}^{3k}.$$

Using a Pedersen commitment key ck_{ABC} for vectors in $\mathbb{F}^{3k}$, the prover samples $o_{ABC,j} \leftarrow \$ \mathbb{F}$ and computes

$$\text{cm}_{ABC,j} \leftarrow \text{Ped.Com}(\text{ck}_{ABC}, \mathbf{m}_{ABC,j}; o_{ABC,j}).$$

The matrix root is the Merkle commitment to the vector of Pedersen commitments:

$$\text{rt}_{ABC} \leftarrow \text{MT.Com}((\text{cm}_{ABC,j})_{j \in [n]}; \perp).$$

The root rt_{ABC} is fixed before the Freivalds challenge r is derived. This ordering fixes the encoded matrix columns before the prover sees the challenge used to form $\mathbf{x}, \mathbf{y}, \mathbf{z}$.

4.3. Intermediate Vectors and Query Sampling

The verifier derives the Freivalds challenge from the matrix root:

$$r \leftarrow \text{Hash}(\text{LAMP-r}, \text{rt}_{ABC}), \quad \mathbf{r} = (1, r, \dots, r^{k-1}).$$

The prover computes

$$\mathbf{x} = \mathbf{r}\mathbf{A}, \quad \mathbf{y} = \mathbf{x}\mathbf{B}, \quad \mathbf{z} = \mathbf{r}\mathbf{C},$$

and encodes the intermediate vectors:

$$\widehat{\mathbf{x}} = \text{Enc}(\mathbf{x}), \quad \widehat{\mathbf{y}} = \text{Enc}(\mathbf{y}), \quad \widehat{\mathbf{z}} = \text{Enc}(\mathbf{z}).$$

For each $j \in [n]$, it packs the encoded vector symbols into

$$\mathbf{m}_{XYZ,j} := (\widehat{\mathbf{x}}[j] \parallel \widehat{\mathbf{y}}[j] \parallel \widehat{\mathbf{z}}[j]) \in \mathbb{F}^3.$$

Using a Pedersen commitment key ck_{XYZ} , the prover samples $o_{XYZ,j} \leftarrow \$ \mathbb{F}$ and computes

$$\text{cm}_{XYZ,j} \leftarrow \text{Ped.Com}(\text{ck}_{XYZ}, \mathbf{m}_{XYZ,j}; o_{XYZ,j}),$$

and publishes the Merkle root

$$\text{rt}_{XYZ} \leftarrow \text{MT.Com}((\text{cm}_{XYZ,j})_{j \in [n]}; \perp).$$

The query tuple is derived only after rt_{XYZ} is fixed:

$$\begin{aligned} \rho_I &\leftarrow \text{Hash}(\text{LAMP-I}, \text{rt}_{ABC}, r, \text{rt}_{XYZ}), \\ I &= (j_1, \dots, j_t) \leftarrow \text{Sample}_t(\rho_I). \end{aligned}$$

The points used for the vector-code consistency checks are derived from the same transcript prefix:

$$\begin{aligned} \rho_{\text{code}} &\leftarrow \text{Hash}(\text{LAMP-code}, \text{rt}_{ABC}, r, \text{rt}_{XYZ}), \\ \alpha_x, \alpha_y, \alpha_z &\leftarrow \text{Derive}_{\mathbb{F}}(\rho_{\text{code}}). \end{aligned}$$

The concrete sampler uses domain separation to obtain field elements outside the code evaluation domain.

4.4. SNARK Relation

The inner SNARK proves the sampled arithmetic relation. Its public statement contains the commitment roots, transcript-derived challenges, query positions, code-check points, and SNARK-side witness commitments:

$$\begin{aligned} \mathbf{x} &= (\text{rt}_{ABC}, \text{rt}_{XYZ}, r, I, \alpha_x, \alpha_y, \alpha_z, \\ &\quad \widetilde{\text{cm}}_{ABC}, \widetilde{\text{cm}}_{XYZ}). \end{aligned}$$

The private witness contains the intermediate vectors, their codewords, and the sampled packed values:

$$\begin{aligned} \mathbf{w} &= (\mathbf{x}, \mathbf{y}, \mathbf{z}, \widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \\ &\quad \{\mathbf{m}_{ABC,j}\}_{j \in I}, \{\mathbf{m}_{XYZ,j}\}_{j \in I}). \end{aligned}$$

The CP-SNARK backend commits to the flattened sampled witnesses $\{\mathbf{m}_{ABC,j}\}_{j \in I}$ and $\{\mathbf{m}_{XYZ,j}\}_{j \in I}$, producing $\widetilde{\text{cm}}_{ABC}$ and $\widetilde{\text{cm}}_{XYZ}$. These commitments are later linked to the external Pedersen leaves.

For every sampled $j \in I$, write

$$\mathbf{m}_{ABC,j} = (\mathbf{a}_j \parallel \mathbf{b}_j \parallel \mathbf{c}_j), \quad \mathbf{m}_{XYZ,j} = (x_j \parallel y_j \parallel z_j),$$

where $\mathbf{a}_j, \mathbf{b}_j, \mathbf{c}_j \in \mathbb{F}^k$ and $x_j, y_j, z_j \in \mathbb{F}$. The SNARK checks:

$$\begin{aligned}
&\hat{\mathbf{x}} \text{ is consistent with } \text{Enc}(\mathbf{x}) \text{ at } \alpha_x, & (\text{CODE-X}) \\
&\hat{\mathbf{y}} \text{ is consistent with } \text{Enc}(\mathbf{y}) \text{ at } \alpha_y, & (\text{CODE-Y}) \\
&\hat{\mathbf{z}} \text{ is consistent with } \text{Enc}(\mathbf{z}) \text{ at } \alpha_z, & (\text{CODE-Z}) \\
&x_j = \hat{\mathbf{x}}[j], \quad y_j = \hat{\mathbf{y}}[j], \quad z_j = \hat{\mathbf{z}}[j], & (\text{LOOKUP}) \\
&x_j = \text{Fold}(\mathbf{r}, \mathbf{a}_j), & (\text{FOLD-A}) \\
&y_j = \text{Fold}(\mathbf{x}, \mathbf{b}_j), & (\text{FOLD-B}) \\
&z_j = \text{Fold}(\mathbf{r}, \mathbf{c}_j), & (\text{FOLD-C}) \\
&y_j = z_j. & (\text{EQ})
\end{aligned}$$

The first three checks enforce that the intermediate code-words used by the sampled relation are encodings of the vector witnesses. The remaining checks are the sampled fold relations from Section 3.2.

4.5. Merkle Openings and CP-Link

The SNARK relation alone does not prove that the sampled witnesses are the values committed in the Merkle trees. The auxiliary verifier therefore checks Merkle openings and CP-Link proofs.

For each $j \in I$, the prover opens the external leaves $\text{cm}_{ABC,j}$ and $\text{cm}_{XYZ,j}$ under rt_{ABC} and rt_{XYZ} :

$$\begin{aligned}
&\text{MT.Verify}(\text{rt}_{ABC}, j, \text{cm}_{ABC,j}, \pi_{ABC,j}) = 1, & (\text{PATH-ABC}) \\
&\text{MT.Verify}(\text{rt}_{XYZ}, j, \text{cm}_{XYZ,j}, \pi_{XYZ,j}) = 1. & (\text{PATH-XYZ})
\end{aligned}$$

It then proves, via CP-Link, that the SNARK-side commitment to the sampled ABC witness opens to the same block messages as the external Pedersen commitments $(\text{cm}_{ABC,j})_{j \in I}$, and similarly for the XYZ sampled witnesses:

$$\begin{aligned}
&\text{CPLink.Verify}(\widetilde{\text{cm}}_{ABC}, (\text{cm}_{ABC,j})_{j \in I}, \pi_{\text{link}}^{ABC}) = 1, & (\text{LINK-ABC}) \\
&\text{CPLink.Verify}(\widetilde{\text{cm}}_{XYZ}, (\text{cm}_{XYZ,j})_{j \in I}, \pi_{\text{link}}^{XYZ}) = 1. & (\text{LINK-XYZ})
\end{aligned}$$

Here CPLink.Verify denotes the batched, blockwise linking relation supplied by the backend: the SNARK-side commitment is opened as the concatenation of the sampled blocks, and each external Pedersen commitment is opened to the corresponding block. Equivalently, it is a batched application of the same-message commitment-linking relation from Section 2.5.

Definition 4.1 (Sampled-opening backend). A sampled-opening backend for LAMP consists of the Merkle opening checks and CP-Link checks above, together with the committed-witness interface of the SNARK. It must satisfy:

- *Position binding*: an accepted Merkle opening fixes a unique Pedersen leaf at the queried position under the root;
- *Commitment binding*: an accepted Pedersen leaf is binding to a unique message, except with negligible probability;
- *SNARK-side binding*: an accepting SNARK proof binds the sampled witnesses used by the circuit to the public SNARK-side commitments $\widetilde{\text{cm}}_{ABC}$ and $\widetilde{\text{cm}}_{XYZ}$;
- *CP-Link consistency*: accepted CP-Link proofs imply that the SNARK-side commitments and the external Pedersen leaves commit to the same sampled messages, except with the CP-Link soundness error.

Under this backend, the verifier obtains the consistency chain

$$\begin{aligned}
&\text{values used in the SNARK} \\
&\Downarrow \text{committed witness and CP-Link} \\
&\quad \text{external Pedersen leaves} \\
&\Downarrow \text{Merkle authentication} \\
&\quad \text{rt}_{ABC}, \text{rt}_{XYZ}.
\end{aligned}$$

This is why the SNARK circuit can avoid in-circuit Pedersen verification and in-circuit Merkle path verification.

4.6. Protocol Algorithms and Complexity

Figure 1 summarizes the non-interactive protocol obtained by Fiat–Shamir. The interactive version is obtained by having the verifier sample r after rt_{ABC} and I after rt_{XYZ} .

The proof consists of the SNARK proof, the sampled Merkle openings, and the CP-Link proofs. The sampled matrix columns and vector symbols themselves are private SNARK witnesses; they are not public proof data. The roots and transcript-derived challenges are public.

We separate three kinds of work. First, the application may need to compute or obtain the claimed output $\mathbf{C}$; this native matrix multiplication cost is shared by all approaches and is not part of the SNARK checking cost. Second, the prover encodes the matrices and builds the matrix commitment root. Third, the online proof checks the committed multiplication statement.

SNARK constraints. For each queried position j , the circuit computes three length- k folds:

$$\begin{aligned}
&\text{Fold}(\mathbf{r}, \hat{\mathbf{A}}[*], j), \\
&\text{Fold}(\mathbf{x}, \hat{\mathbf{B}}[*], j), \\
&\text{Fold}(\mathbf{r}, \hat{\mathbf{C}}[*], j).
\end{aligned}$$

Thus the sampled fold constraints cost $\mathcal{O}(tk)$. The vector encoding checks for $\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}}$ are also linear in $k + n$ per checked out-of-domain point for the Reed–Solomon instantiation used in the artifact. For constant-rate codes and fixed t , the dominant in-circuit checking cost is linear in k .

Auxiliary work. Merkle openings contribute $\mathcal{O}(t \log n)$ hash work and proof material. CP-Link contributes backend-dependent proving and verification costs, denoted by $E_{\text{link}}(k, t)$. These checks are outside the SNARK circuit.

Comparison. Naïve SNARK arithmetization of matrix multiplication costs $\mathcal{O}(k^3)$ constraints. A Freivalds-in-SNARK baseline computes the vector-matrix products directly and costs $\mathcal{O}(k^2)$ constraints. LAMP replaces the full vector-matrix computation with sampled encoded-column checks, giving $\mathcal{O}(tk)$ fold constraints plus auxiliary opening and linking work.

5. Security Analysis

We prove that LAMP is sound for the committed matrix multiplication statement defined in Section 4. The public roots rt_{ABC} and rt_{XYZ} bind, respectively, to the packed

Public parameters: field $\mathbb{F}$, code $\mathcal{C}$, query count t , Pedersen keys $\text{ck}_{ABC}, \text{ck}_{XYZ}$, Merkle hash, SNARK parameters, and CP-Link parameters.

Prover $\mathcal{P}(\mathbf{A}, \mathbf{B}, \mathbf{C})$.

1. Encode $\mathbf{A}, \mathbf{B}, \mathbf{C}$ row-wise. Build $\text{cm}_{ABC,j}$ and rt_{ABC} .
2. Derive r and set $\mathbf{r} = (1, r, \dots, r^{k-1})$. Compute $\mathbf{x} = \mathbf{r}\mathbf{A}$, $\mathbf{y} = \mathbf{x}\mathbf{B}$, and $\mathbf{z} = \mathbf{r}\mathbf{C}$. Encode $\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}}$.
3. Build $\text{cm}_{XYZ,j}$ and rt_{XYZ} . Derive the query tuple I and code-check points $\alpha_x, \alpha_y, \alpha_z$.
4. For every $j \in I$, prepare the sampled witnesses $\mathbf{m}_{ABC,j}$ and $\mathbf{m}_{XYZ,j}$, and the corresponding Merkle authentication paths.
5. Generate the CP-SNARK proof for the relation in Section 4.4. Extract the SNARK-side witness commitments $\widetilde{\text{cm}}_{ABC}$ and $\widetilde{\text{cm}}_{XYZ}$.
6. Generate CP-Link proofs π_{link}^{ABC} and π_{link}^{XYZ} . Output the roots, SNARK proof, Merkle openings, and CP-Link proofs.

Verifier $\mathcal{V}$.

1. Recompute r , I , and $\alpha_x, \alpha_y, \alpha_z$ from the transcript.
2. Verify the SNARK proof against the public statement $\mathbf{x}$.
3. Verify all Merkle openings against rt_{ABC} and rt_{XYZ} .
4. Verify the two CP-Link proofs connecting SNARK-side commitments to the opened Pedersen leaves.
5. Accept iff all checks accept.

Figure 1. The LAMP protocol for verifiable matrix multiplication.

Component	Prover	Verifier
Matrix encoding/root	$\mathcal{O}(kn)$	roots fixed before r
Intermediate vectors	$\mathcal{O}(k^2)$ native work	—
Vector encoding/root	$\mathcal{O}(n)$ for constant-rate codes	—
SNARK circuit	$\mathcal{O}(tk)$ constraints	SNARK verification
Merkle openings	$\mathcal{O}(t \log n)$	$\mathcal{O}(t \log n)$
CP-Link	$E_{\text{link}}(k, t)$	backend-dependent

TABLE 2. ASYMPTOTIC COSTS FOR LAMP. THE TABLE SEPARATES NATIVE MATRIX COMPUTATION AND COMMITMENT GENERATION FROM THE IN-CIRCUIT CHECKING RELATION.

encoded matrix columns and the packed encoded symbols of the intermediate vectors. The sampled-opening backend of Definition 4.1 links the sampled witnesses used by the SNARK to the corresponding opened Pedersen leaves. Completeness and soundness are stated in Theorem 5.1; knowledge soundness is discussed separately in Remark 3.

Scope of the security claim. The theorem is a statement-soundness result for committed matrix multiplication. It does not, by itself, claim a standalone argument of knowledge or zero knowledge for the entire input-commitment layer. The committed objects are:

$$\text{rt}_{ABC} \text{ and } \text{rt}_{XYZ},$$

where rt_{ABC} fixes the encoded matrix-column blocks $\{\mathbf{m}_{ABC,j}\}_{j \in [n]}$, and rt_{XYZ} fixes the encoded intermediate-vector blocks $\{\mathbf{m}_{XYZ,j}\}_{j \in [n]}$. The SNARK-side commitments $\widetilde{\text{cm}}_{ABC}$ and $\widetilde{\text{cm}}_{XYZ}$ bind the sampled blocks used inside the SNARK. CP-Link then links these sampled blocks to the Pedersen leaves opened under rt_{ABC} and rt_{XYZ} .

Soundness-critical ordering. The proof relies on the transcript order enforced by the construction. First, rt_{ABC} is fixed before the Freivalds scalar r is derived. Thus the encoded matrices are fixed before the prover sees the challenge used to define $\mathbf{x} = \mathbf{r}\mathbf{A}$, $\mathbf{y} = \mathbf{x}\mathbf{B}$, and $\mathbf{z} = \mathbf{r}\mathbf{C}$. Second, rt_{XYZ} is fixed before the query tuple $I = (j_1, \dots, j_t)$ is derived. Thus the encoded intermediate-vector symbols are fixed before the sampled positions are known. Third, the verifier accepts sampled values only when the SNARK proof, Merkle openings, and CP-Link proofs all accept. Conditioned on the soundness of these components, the values used in the sampled fold checks are exactly the values fixed by rt_{ABC} and rt_{XYZ} at the queried positions.

Backend failure event. The backend bad event captures the only composition point specific to our construction. After the transcript prefix fixes

$$(\text{rt}_{ABC}, r, \text{rt}_{XYZ}, I),$$

the adversary outputs a SNARK proof, sampled Merkle openings, and CP-Link proofs for the two sampled block families. A backend failure occurs if all these checks accept while either $\widetilde{\text{cm}}_{ABC}$ is not linked to the opened blocks $(\text{cm}_{ABC,j})_{j \in I}$, or $\widetilde{\text{cm}}_{XYZ}$ is not linked to the opened blocks $(\text{cm}_{XYZ,j})_{j \in I}$, or an opened Merkle leaf is not the value fixed at the claimed position under its root. Definition 4.1 charges this event to $\epsilon_{\text{open}} + \epsilon_{\text{link}}$, where ϵ_{open} covers Merkle position binding and Pedersen binding, and ϵ_{link} covers CP-Link soundness.

Theorem 5.1 (Security). *The protocol LAMP satisfies the following properties.*

- (i) **Perfect completeness.** *If $\mathbf{AB} = \mathbf{C}$, an honest prover makes the verifier accept.*
- (ii) **Interactive computational soundness.** *For any PPT adversary $\mathcal{P}^*$, let Acc be the event that the verifier*

accepts a false committed matrix multiplication statement. Then

$$\Pr[\text{Acc} \mid \mathbf{AB} \neq \mathbf{C}] \leq \epsilon_{\text{SNARK}}(\lambda) + \epsilon_{\text{open}}(\lambda) + \epsilon_{\text{link}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta)^t.$$

Here $t = |I|$ is the number of sampled positions and δ is the relative distance of the code $\mathcal{C}$. The term ϵ_{SNARK} is the soundness error of the CP-SNARK relation, ϵ_{open} is the binding error of the Merkle/Pedersen opening layer, and ϵ_{link} is the soundness error of the two block-wise CP-Link checks.

- (iii) **Fiat–Shamir soundness.** In the random oracle model, the non-interactive protocol in Section 4.6 satisfies the same bound with an additional Fiat–Shamir loss $\epsilon_{\text{FS}}(q_H)$, where q_H is the adversary’s number of random-oracle queries.

Corollary 5.2 (Evaluated backend). *For the backend used in the artifact, the opening and linking contribution in Theorem 5.1 can be written as*

$$\epsilon_{\text{open}} + \epsilon_{\text{link}} \leq \epsilon_{\text{merkle}} + \epsilon_{\text{ped}} + \epsilon_{\text{QA}} + \frac{2t}{|\mathbb{F}|}.$$

The first two terms cover Merkle position binding and Pedersen binding. The last two terms cover the batched CP-Link proof for the two sampled block families, ABC and XYZ .

The reported scaling rows use $t = 128$. If $\delta = 1/2$, the proximity term is $4(1-\delta)^t = 2^{-126}$. A deployment that targets a different security level can increase t , choose a larger-distance code, or instantiate the backend with smaller component errors.

Proof sketch. Completeness follows from linearity of the Freivalds reduction and of the encoding. Honest commitments, openings, and CP-Link proofs make every sampled fold and equality check accept.

For soundness, condition away SNARK, opening, and CP-Link failures. Then rt_{ABC} fixes the encoded matrices before r , and rt_{XYZ} fixes the encoded intermediate-vector symbols before I . The sampled values used inside the SNARK are the same values opened from these roots. If $\mathbf{AB} \neq \mathbf{C}$, the structured Freivalds check fails to detect the false product with probability at most $(k-1)/|\mathbb{F}|$. Conditioned on not hitting this Freivalds failure event, at least one of the local encoded relations checked in Section 4.4 is false. By the relative distance δ , a random sampled position misses the disagreement with probability at most $1-\delta$. Union-bounding over the four checked relations gives the $4(1-\delta)^t$ term. Adding the SNARK, opening, linking, and Fiat–Shamir failure events gives the stated bound.

Remark 2 (Fiat–Shamir soundness). The theorem is stated first for the public-coin interactive protocol. The non-interactive protocol derives r , I , and backend batch challenges from domain-separated hashes of the transcript. In the random oracle model, the same argument applies with the standard Fiat–Shamir loss $\epsilon_{\text{FS}}(q_H)$.

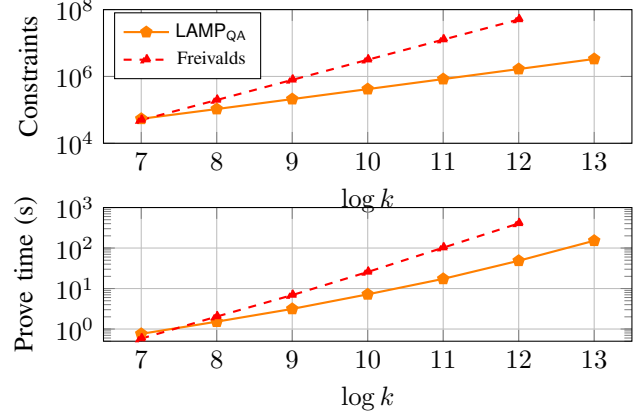


Figure 2. Primary matrix benchmark. With the sampled-opening backend, LAMP_{QA} follows the expected $\mathcal{O}(k)$ checking-layer growth, whereas the Freivalds SNARK baseline follows $\mathcal{O}(k^2)$ growth.

Remark 3 (Knowledge Soundness). The theorem above is a statement-soundness claim relative to the committed roots rt_{ABC} and rt_{XYZ} . A full argument of knowledge can be obtained by composing LAMP with an extractable commitment interface and a knowledge-sound CP-SNARK. This is an orthogonal composition issue; the main theorem focuses on soundness of the matrix-multiplication checking protocol.

6. Implementation and Evaluation

We implement the multiplication-checking layer of LAMP and evaluate the current artifact, denoted LAMP_{QA}, against an optimized SNARK-in-circuit Freivalds baseline [1]. LAMP_{QA} uses multi-column Merkle openings, a Groth16 circuit proof, and the sampled-opening backend from Section 4. All main experiments use a 256-bit prime field, rate $\rho = 1/2$, codeword length $n = 2k$, and $t = 128$ sampled positions. The experiments were run on a Linux server with an AMD EPYC 7B13 CPU (32 cores) and 254GiB RAM.

These measurements include the online sampled-handle path described in Section 4: rt_{ABC} fixes the packed encoded matrix columns before r , rt_{XYZ} fixes the packed encoded vector symbols before I , the CP-SNARK backend binds the sampled blocks to $\widehat{\mathbf{cm}}_{ABC}$ and $\widehat{\mathbf{cm}}_{XYZ}$, Merkle openings authenticate the opened Pedersen leaves, and blockwise CP-Link connects the SNARK-side sampled blocks to those leaves. The raw sampled matrix columns are SNARK witness data, not serialized auxiliary proof bytes. All byte counts in this section are online proof bytes: they exclude raw-matrix encoding and construction of rt_{ABC} , proving/verifying keys, serialized CRS data, and generator material. They should not be read as complete end-to-end deployment costs from raw matrices. Appendix C records parameter and measurement coverage details.

Primary matrix benchmark. Figure 2 shows the main scaling behavior. Freivalds constraints grow by approximately $4\times$ with each doubling of k , matching the expected

$\log k$	Constraints		Prove Time (s)		Verify Time		LAMP_{QA}	Online Proof
	LAMP _{QA}	Freivalds	LAMP _{QA}	Freivalds	LAMP _{QA}	Freivalds		
7	54,069	49,610	0.76	0.58	0.01 s	1 ms	22,404 B	
8	105,915	197,194	1.52	2.03	0.01 s	1 ms	29,188 B	
9	209,995	788,810	3.14	6.89	0.01 s	1 ms	35,204 B	
10	418,149	3,156,298	7.19	25.63	0.01 s	2 ms	44,292 B	
11	834,075	12,622,154	17.33	102.07	0.01 s	2 ms	51,012 B	
12	1,666,315	50,495,818	48.54	405.04	0.01 s	2 ms	59,332 B	
13	3,330,789	—	151.61	—	0.01 s	—	67,268 B	

TABLE 3. PRIMARY MATRIX-MULTIPLICATION BENCHMARK FOR **LAMP_{QA}** AND THE OPTIMIZED **FREIVALDS SNARK** BASELINE. **LAMP_{QA}** IS THE CURRENT IMPLEMENTATION OF THE SAMPLED-OPENING BACKEND WITH MULTI-COLUMN MERKLE OPENINGS. PROVING TIME EXCLUDES NATIVE MATRIX MULTIPLICATION AND SETUP; THESE ARE RECORDED SEPARATELY IN THE ARTIFACT. THE TABLE MEASURES THE ONLINE MULTIPLICATION-CHECKING LAYER, INCLUDING SAMPLED RS EVALUATIONS, MERKLE OPENINGS, SNARK-SIDE WITNESS COMMITMENTS, AND AUXILIARY OPENING/LINKING CHECKS; ONLINE PROOF BYTES EXCLUDE RAW SAMPLED MATRIX-COLUMN WITNESS VALUES, RAW-MATRIX ENCODING AND INPUT-ROOT CONSTRUCTION FOR CM_A , CM_B , CM_C , AND SETUP/KEY/CRS MATERIAL.

$\log k$	Commit Time	Proving Time			Online Proof Size			Verification Time		
		SNARK	Merkle	CP-Link	SNARK	Merkle	CP-Link	SNARK	Merkle	CP-Link
7	0.10 s	0.63 s	0 ms	0.02 s	260 B	22,016 B	128 B	2 ms	1 ms	3 ms
8	0.29 s	1.18 s	0 ms	0.05 s	260 B	28,800 B	128 B	2 ms	1 ms	3 ms
9	0.91 s	2.17 s	0 ms	0.07 s	260 B	34,816 B	128 B	2 ms	1 ms	3 ms
10	3.20 s	3.88 s	0 ms	0.12 s	260 B	43,904 B	128 B	2 ms	1 ms	3 ms
11	10.02 s	7.10 s	0 ms	0.21 s	260 B	50,624 B	128 B	2 ms	1 ms	3 ms
12	34.57 s	13.55 s	0 ms	0.42 s	260 B	58,944 B	128 B	2 ms	2 ms	3 ms
13	123.87 s	26.81 s	0 ms	0.93 s	260 B	66,880 B	128 B	2 ms	2 ms	3 ms

TABLE 4. **LAMP_{QA}** COMPONENT BREAKDOWN FROM `MEOW_BENCHMARK_RESULTS_V2.CSV`. COMMIT TIME IS `MATRIXCOMMITTIME` PLUS `VECTORCOMMITTIME`. SNARK PROVING TIME IS `CIRCUITPROVE`; SNARK PROOF SIZE IS THE GROTH16 PROOF SIZE. MERKLE AND CP-LINK COLUMNS REPORT THE CORRESPONDING PROVING, PROOF-SIZE, AND VERIFICATION COMPONENTS FROM THE ARTIFACT.

Metric	LAMP_{QA}	Freivalds	Ratio
Constraints	33.4M	76.8M	0.43×
Setup time	2,078.38 s	4,173.32 s	0.50×
Prove time	230.14 s	408.69 s	0.56×
Verify time	0.68 s	0.49 s	1.39×
LAMP_{QA} online proof	3,355,428 B	—	—

TABLE 5. EXACT GPT-2 CHECKING-LAYER METRICS FROM THE CSV ARTIFACT FOR SEQUENCE LENGTH 1,024, 36 MATRIX-MULTIPLICATION CLAIMS, AND 5 COMMITMENT GROUPS.

$\mathcal{O}(k^2)$ scaling. In contrast, **LAMP_{QA}** grows by approximately $2\times$, matching the $\mathcal{O}(k)$ checking-layer scaling of the sampled-opening backend. The crossover in absolute constraint count occurs at $k = 256$. At $k = 1,024$, **LAMP_{QA}** uses 418,149 constraints versus 3,156,298 for Freivalds, a $7.55\times$ reduction. At $k = 4,096$, **LAMP_{QA}** uses 1,666,315 constraints versus 50,495,818 for Freivalds, a $30.30\times$ reduction.

The same benchmark shows a proving-time crossover at $k = 256$. At $k = 4,096$, **LAMP_{QA}** proves in 48.54s versus 405.04s for Freivalds, an $8.34\times$ speedup. **LAMP_{QA}** also completes $k = 8,192$ with 3,330,789 constraints and a 151.61s proving time; Freivalds was not measured at that size in the current artifact. Table 4 breaks down the **LAMP_{QA}** rows into commitment, SNARK, Merkle, and CP-Link components.

Verification, online bytes, and GPT-2 workload. **LAMP_{QA}** verification is nearly constant at about 0.01s in the matrix benchmark, dominated by the auxiliary linking backend; Freivalds verification is faster because it checks

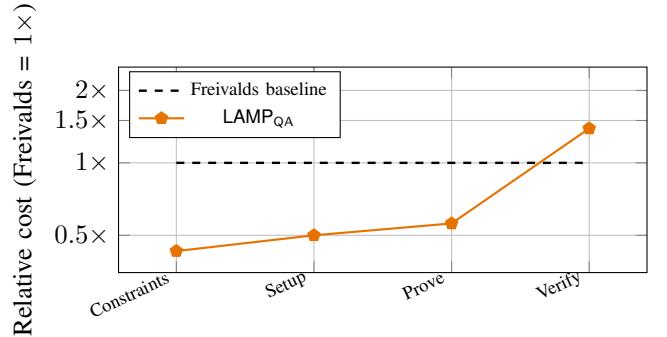


Figure 3. GPT-2 checking-layer case study at sequence length 1,024 with 36 matrix-multiplication claims over committed roots. Costs are normalized to the Freivalds baseline.

only a direct Groth16 proof. The artifact therefore trades slower verification for smaller prover circuits and faster proving at medium and large dimensions. The **LAMP_{QA}** online proof grows from 22,404 bytes at $k = 128$ to 59,332 bytes at $k = 4,096$ and 67,268 bytes at $k = 8,192$. At $k = 4,096$, $t = 128$, publishing the three sampled input-matrix columns alone would require about $3tk \cdot 32 \approx 50$ MB; Appendix C gives detailed byte, setup, and constraint accounting.

Figure 3 and Table 5 report the GPT-2-style workload over committed roots. **LAMP_{QA}** reduces constraints by $2.30\times$ and proving time by $1.78\times$, with lower setup time in this experiment. Verification is $1.39\times$ slower, so latency-

sensitive deployments still need batching, amortization, or a more succinct opening/linking backend. The 3.36 MB GPT-2 online proof is larger than the single-matrix rows because it aggregates 36 multiplication claims and their sampled SNARK-side commitments, Merkle authentication paths, and linking material.

Strict parameters and deployment scope. The measured rows use $t = 128$ to study scaling. If $\delta = 1/2$, a conservative per-term 2^{-131} proximity budget uses $t = 133$, increasing sample-dependent work by only $1.0391\times$. At $k = 4,096$, this extrapolates to at most 1.74 million constraints, 50.44 s proving time, and 62 KB online proof bytes. The current artifact does not include measured $t = 133$ rows; these strict numbers are extrapolations from the measured $t = 128$ rows. Online-byte figures exclude raw-matrix encoding, committed-root construction, proving/verifying keys, and serialized CRS or generator material. End-to-end deployments preserve the measured prover-time win only if amortized root generation costs stay below the observed margins: 356.50 s for $k = 4,096$ and 178.55 s for the GPT-2 row. The CSV artifact reports point estimates and aggregate setup/proving/verification times; repeated-run variance, confidence intervals, peak memory, and serialized key/CRS/generator sizes remain unmeasured.

7. Conclusion and Related Work

Our work complements verifiable ML systems such as SafetyNets [4], Mystique [2], zkCNN [10], vCNN [11], and zkML [3]; sumcheck/GKR-based VC [12]; dedicated matrix-verification protocols [13]–[16]; and code-based SNARK/proximity systems [5], [17]–[19]. Unlike full-circuit ML systems or polynomial-commitment matrix arguments [20], LAMP is a dedicated multiplication checking layer over committed encoded data and is orthogonal to succinct backends, including universal/updatable-SRS and inner-product-style systems [21]–[23]. The security theorem accounts for SNARK soundness, opening and linking soundness, Freivalds failure, and proximity-sampling error.

The artifact confirms the expected separation. At $k = 4,096$, LAMP_{QA} gives a $30.29\times$ constraint reduction and an $8.34\times$ proving-time speedup over Freivalds-in-SNARK; on the GPT-2 case study it gives a $2.30\times$ constraint reduction and a $1.78\times$ proving-time speedup over committed roots. The remaining limitation is explicit: the artifact measures the online checking layer after committed-root generation. Reporting raw-matrix encoding/root costs, key/CRS sizes, reducing verification overhead, and batching many multiplications are the main deployment steps.

References

- [1] R. Freivalds, “Probabilistic machines can use less running time,” in *Information Processing 77 (Proceedings of IFIP Congress 77)*. Amsterdam, The Netherlands: North-Holland, 1977, pp. 839–842.
- [2] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, “Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning,” in *30th USENIX Security Symposium (USENIX Security 21)*. Berkeley, CA, USA: USENIX Association, 2021, pp. 501–518.
- [3] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, “Zkml: An optimizing system for ml inference in zero-knowledge proofs,” in *Proceedings of the Nineteenth European Conference on Computer Systems*. New York, NY, USA: ACM, 2024, pp. 560–574.
- [4] Z. Ghodsi, T. Gu, and S. Garg, “Safetynets: Verifiable execution of deep neural networks on an untrusted cloud,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, “Brakedown: Linear-time and field-agnostic snarks for r1cs,” in *Advances in Cryptology – CRYPTO 2023*. Cham: Springer, 2023, pp. 193–226.
- [6] A. Belling, A. Soleimani, and O. Bégassat, “Recursion over public-coin interactive proof systems; faster hash verification,” *Cryptology ePrint Archive*, Paper 2022/1072, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1072>
- [7] M. Campanelli, D. Fiore, and A. Querol, “LegoSNARK: Modular design and composition of succinct zero-knowledge proofs,” *Cryptology ePrint Archive*, Paper 2019/142, 2019. [Online]. Available: <https://eprint.iacr.org/2019/142>
- [8] J. T. Schwartz, “Fast probabilistic algorithms for verification of polynomial identities,” *Journal of the ACM*, vol. 27, no. 4, pp. 701–717, 1980.
- [9] R. Zippel, “Probabilistic algorithms for sparse polynomials,” in *Symbolic and Algebraic Computation (EUROSAM ’79)*. Berlin, Heidelberg: Springer, 1979, pp. 216–226.
- [10] T. Liu, X. Xie, and Y. Zhang, “Zkcn: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2021, pp. 2968–2985.
- [11] S. Lee, H. Ko, J. Kim, and H. Oh, “vcnn: Verifiable convolutional neural network based on zk-snarks,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4254–4270, 2024.
- [12] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, “Delegating computation: Interactive proofs for muggles,” *Journal of the ACM*, vol. 62, no. 4, pp. 1–64, 2015.
- [13] M. Cong, T. H. Yuen, and S.-M. Yiu, “zkmatrix: Batched short proof for committed matrix multiplication,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2024, pp. 289–305.
- [14] M. Cong, T. H. Yuen, and S. M. Yiu, “Dualmatrix: conquering zkcn for large matrix multiplication,” *Cybersecurity*, vol. 9, no. 1, p. 126, 2026.
- [15] Y. Zhang, M. Zheng, X. Chen, J. Hu, W. Shi, L. Ju, Y. Solihin, and Q. Lou, “zkvc: Fast zero-knowledge proof for private and verifiable computing,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, IEEE. New York, NY, USA: IEEE, 2025, pp. 1–7.
- [16] H. Bennett, K. Gajulapalli, A. Golovnev, and E. Warton, “Matrix multiplication verification using coding theory,” *arXiv preprint arXiv:2309.16176*, 2023.
- [17] S. Ames, C. Hazay, Y. Ishai, and M. Venkatasubramanian, “Ligero: Lightweight sublinear arguments without a trusted setup,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2017, pp. 2087–2104.
- [18] T. Xie, Y. Zhang, and D. Song, “Orion: Zero knowledge proof with linear prover time,” in *Advances in Cryptology – CRYPTO 2022*. Cham: Springer, 2022, pp. 299–328.
- [19] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast reed-solomon interactive oracle proofs of proximity,” in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 14:1–14:17.

- [20] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-size commitments to polynomials and their applications,” in *Advances in Cryptology – ASIACRYPT 2010*. Berlin, Heidelberg: Springer, 2010, pp. 177–194.
- [21] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn, “Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2019, pp. 2111–2128.
- [22] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, “Marlin: Preprocessing zkSNARKs with universal and updatable SRS,” in *Advances in Cryptology – EUROCRYPT 2020*. Cham: Springer, 2020, pp. 738–768.
- [23] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, “Bulletproofs: Short proofs for confidential transactions and more,” in *2018 IEEE Symposium on Security and Privacy*. New York, NY, USA: IEEE, 2018, pp. 315–334.

Appendix A.

Detailed Proof of Theorem 5.1

Proof. We prove the public-coin interactive protocol first. The Fiat–Shamir variant is handled at the end.

Perfect completeness. Assume that the honest prover holds matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ with $\mathbf{AB} = \mathbf{C}$. Let $\mathbf{r} = (1, r, \dots, r^{k-1})$, and define

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}.$$

Then $\mathbf{y} = \mathbf{z}$. The honest prover commits to the packed encoded matrix blocks $\{\mathbf{m}_{ABC,j}\}_{j \in [n]}$ under $\mathbf{rt}_{ABC}$, commits to the packed encoded vector blocks $\{\mathbf{m}_{XYZ,j}\}_{j \in [n]}$ under $\mathbf{rt}_{XYZ}$, and supplies the sampled openings and CP-Link proofs required by Section 4.5.

For every queried position $j \in I$, code linearity gives

$$\text{Fold}(\mathbf{r}, \hat{\mathbf{A}}[\ast, j]) = \hat{\mathbf{x}}[j],$$

and similarly

$$\text{Fold}(\mathbf{x}, \hat{\mathbf{B}}[\ast, j]) = \hat{\mathbf{y}}[j], \quad \text{Fold}(\mathbf{r}, \hat{\mathbf{C}}[\ast, j]) = \hat{\mathbf{z}}[j].$$

Since $\mathbf{y} = \mathbf{z}$, the equality check $\hat{\mathbf{y}}[j] = \hat{\mathbf{z}}[j]$ also holds. All SNARK, Merkle, and CP-Link checks are honestly generated, so the verifier accepts.

Bad-event decomposition. Let Acc be the event that the verifier accepts a false committed matrix multiplication statement. We decompose the failure probability into the following events:

- $\text{Bad}_{\text{SNARK}}$: the CP-SNARK accepts although the sampled relation of Section 4.4 is false for the committed sampled witnesses;
- Bad_{open} : a Merkle opening or Pedersen leaf opening is accepted in a way that violates position binding or commitment binding;
- Bad_{link} : a CP-Link proof accepts although the SNARK-side sampled blocks are not the same messages as the opened Pedersen leaves under $\mathbf{rt}_{ABC}$ and $\mathbf{rt}_{XYZ}$.

By the assumed soundness of the selected backends,

$$\begin{aligned} \Pr[\text{Bad}_{\text{SNARK}}] &\leq \epsilon_{\text{SNARK}}(\lambda), \\ \Pr[\text{Bad}_{\text{open}}] &\leq \epsilon_{\text{open}}(\lambda), \\ \Pr[\text{Bad}_{\text{link}}] &\leq \epsilon_{\text{link}}(\lambda). \end{aligned}$$

Condition on the complement of these events. Then $\mathbf{rt}_{ABC}$ fixes a unique sequence of packed matrix-column commitments at all queried positions, $\mathbf{rt}_{XYZ}$ fixes a unique sequence of packed vector-symbol commitments at all queried positions, and the sampled values used by the SNARK are exactly the messages opened from those commitments. In particular, the adversary cannot choose different sampled blocks after seeing I without breaking the opening or linking layer.

Freivalds randomization. The matrix root $\mathbf{rt}_{ABC}$ is fixed before r is derived. In the committed statement, $\mathbf{rt}_{ABC}$ binds to row-wise encoded matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$. Suppose $\mathbf{AB} \neq \mathbf{C}$, and let $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$. Some column $\mathbf{d}$ of $\mathbf{D}$ is nonzero. The event $\mathbf{rD} = \mathbf{0}$ implies

$$P(r) := \langle (1, r, \dots, r^{k-1}), \mathbf{d} \rangle = \sum_{i=0}^{k-1} d_i r^i = 0.$$

The polynomial P is nonzero and has degree at most $k-1$. Since r is sampled after $\mathbf{rt}_{ABC}$ is fixed, Schwartz–Zippel gives

$$\Pr[\mathbf{rD} = \mathbf{0}] \leq \frac{k-1}{|\mathbb{F}|}.$$

Proximity sampling. Now also condition on $\mathbf{rD} \neq \mathbf{0}$. No vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ can satisfy all four global relations

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}, \quad \mathbf{y} = \mathbf{z}.$$

Otherwise they would imply $\mathbf{rAB} = \mathbf{rC}$, contradicting $\mathbf{rD} \neq \mathbf{0}$.

Therefore at least one checked encoded relation is false. Consider the case $\mathbf{x} \neq \mathbf{rA}$; the other three cases are identical. The codewords $\text{Enc}(\mathbf{x})$ and $\text{Enc}(\mathbf{rA})$ are distinct, so by relative distance they disagree on at least δn coordinates. For any queried coordinate j , code linearity gives

$$\text{Enc}(\mathbf{rA})[j] = \text{Fold}(\mathbf{r}, \hat{\mathbf{A}}[\ast, j]).$$

The sampled-opening backend ensures that the SNARK uses the opened $\hat{\mathbf{A}}[\ast, j]$ block under $\mathbf{rt}_{ABC}$ and the opened $\hat{\mathbf{x}}[j]$ symbol under $\mathbf{rt}_{XYZ}$. Thus the fold check rejects whenever j lands in the disagreement set. A uniformly sampled coordinate misses this set with probability at most $1 - \delta$, and t independent samples miss it with probability at most $(1 - \delta)^t$. Union-bounding over the four possible violated relations gives

$$4(1 - \delta)^t.$$

Combining the bounds. Adding the SNARK, opening, linking, Freivalds, and proximity failure events gives

$$\begin{aligned} \Pr[\text{Acc} \mid \mathbf{AB} \neq \mathbf{C}] &\leq \epsilon_{\text{SNARK}}(\lambda) + \epsilon_{\text{open}}(\lambda) + \epsilon_{\text{link}}(\lambda) \\ &\quad + \frac{k-1}{|\mathbb{F}|} + 4(1 - \delta)^t. \end{aligned}$$

For the measured rows, $t = 128$. When $\delta = 1/2$, the proximity term is $4 \cdot 2^{-128} = 2^{-126}$. Larger t or a larger-distance code can be used when the deployment target requires a smaller proximity term.

Fiat–Shamir variant. The non-interactive protocol derives r , I , and backend batch challenges from domain-separated hashes of the transcript. In the random oracle model, the public-coin proof above applies with the standard Fiat–Shamir loss $\epsilon_{FS}(q_H)$ for an adversary making q_H random-oracle queries. This proves Theorem 5.1. $\square$

Appendix B. CP-Link Backend Instantiation

This appendix records the concrete blockwise CP-Link relation used by the artifact. For each queried $j \in I$, the external Pedersen leaves have the form

$$\begin{aligned} \text{cm}_{ABC,j} &= \text{Ped.Com}(\text{ck}_{ABC}, \mathbf{m}_{ABC,j}; o_{ABC,j}), \\ \text{cm}_{XYZ,j} &= \text{Ped.Com}(\text{ck}_{XYZ}, \mathbf{m}_{XYZ,j}; o_{XYZ,j}). \end{aligned}$$

The CP-SNARK committed-witness interface produces $\widetilde{\text{cm}}_{ABC}$ for the concatenated sampled matrix blocks $(\mathbf{m}_{ABC,j})_{j \in I}$, and $\widetilde{\text{cm}}_{XYZ}$ for the concatenated sampled vector blocks $(\mathbf{m}_{XYZ,j})_{j \in I}$. The CP-Link proof shows that these SNARK-side commitments and the opened external Pedersen leaves contain the same block messages.

Theorem B.1 (Batched CP-Link backend). *Assume the Pedersen parameters are binding, the QA-NIZK proof for the linear batch relation is sound, and the batching challenge is sampled by the Fiat–Shamir random oracle after the batch statement is fixed. For the two sampled block families ABC and XYZ , each with t queried blocks, the batched CP-Link verifier accepts a false batch with probability at most*

$$\epsilon_{\text{ped}} + \epsilon_{\text{QA}} + \frac{2t}{|\mathbb{F}|}.$$

Generator and CRS bytes are setup material and are not included in the online proof-size claim.

The logical CP-Link statement contains one equality per opened block. We index these $Q = 2t$ equalities by $q \in [Q]$. Each equality has the form

$$C_q^{\text{snark}} - C_q^{\text{leaf}} = \Delta_q H,$$

where C_q^{snark} is the commitment induced by the SNARK-side sampled-block commitment, C_q^{leaf} is the corresponding external Pedersen leaf commitment, and Δ_q is the difference of the two opening randomness values. The batch statement is

$$\begin{aligned} P_I &= \{(\text{tag}_q, C_q^{\text{snark}}, C_q^{\text{leaf}})\}_{q=1}^Q, \\ T_I &= \text{Hash}(\text{LAMP-link-batch}, \text{rt}_{ABC}, \text{rt}_{XYZ}, I, P_I). \end{aligned}$$

The implementation derives

$$\beta \leftarrow \text{Hash}(\text{LAMP-link-rand}, T_I, P_I)$$

and checks the single randomized linear relation

$$\sum_{q=1}^Q \beta^q (C_q^{\text{snark}} - C_q^{\text{leaf}}) = \left(\sum_{q=1}^Q \beta^q \Delta_q \right) H.$$

If any individual equality is false after the batch statement is fixed, the left-minus-right side is a nonzero polynomial in β of degree at most Q . Hence the randomized batch can pass with probability at most $Q/|\mathbb{F}| = 2t/|\mathbb{F}|$. Combining this batch error with Pedersen binding and QA-NIZK soundness gives Theorem B.1. The reported 128-byte CP-Link proof is one online batched QA-NIZK proof for the whole sampled batch, not a per-link cost.

Appendix C. Additional Evaluation Details

This appendix records the measurement boundary behind the main evaluation. Tables 6 and 7 first state the fixed experimental parameters and which costs are covered by the artifact. Tables 8–11 then separate serialized proof material from private Groth16 witness data and from costs that remain unmeasured, such as raw-matrix encoding/root construction and serialized key/CRS size. The intent is to make the main text’s proof-size and speedup claims auditable without treating the checking-layer measurements as full end-to-end VC costs.

Table 6 fixes the common parameter choices used by the reported rows. In particular, the measured rows use $t = 128$ for scaling, while the table also records the nearby query counts needed for stricter proximity-only and per-term security budgets.

Parameter	Value in main experiments
Field size	256-bit prime field
Server	AMD EPYC 7B13, 32 cores, 251.69 GiB RAM, Linux/amd64
Matrix dimensions	$\text{LAMP}_{\text{QA}}: 2^7 \cdot 2^{13}$; Freivalds: $2^7 \cdot 2^{12}$
Code rate	$\rho = 1/2$
Codeword length	$n = 2k$
Sampled positions	$t = 128$
Primary linking layer	Packed Merkle roots, SNARK-side sampled-block commitments, QA-NIZK/CP-Link, and multi-column Merkle openings
Backend model	CP-Link sampled intermediate backend; input roots are fixed before r
Query-count targets	Measured rows use $t = 128$; proximity-only 2^{-128} at $\delta = 1/2$ uses $t = 130$; conservative per-term 2^{-131} uses $t = 133$.

TABLE 6. CONCRETE PARAMETERS USED IN THE MAIN EXPERIMENTS.

Table 7 is a measurement-scope checklist rather than a performance result. It distinguishes quantities measured directly by the CSV artifact from costs that remain outside the online proof bytes, most notably raw-matrix encoding/root construction and serialized key/CRS sizes.

Metric	Covered?	Interpretation
Constraint count	Yes, aggregate	Groth16 circuit size including sampled RS evaluations and fold checks; per-gadget constraint counters are not reported, and CP-SNARK/linking overhead is measured in time/proof bytes rather than as extra R1CS constraints.
Online proving time	Yes	Excludes native matrix multiplication and one-time setup.
Verification time	Yes	Includes current QA-NIZK/CP-Link and opening verification overhead.
Proof size	Yes	Includes Groth16, compact sampled commitments/openings, and QA-NIZK material; raw sampled matrix columns are Groth16 witness data, not serialized proof bytes.
Setup time	Partial	Recorded for the matrix and GPT-2 workloads; setup/key sizes are not reported.
Committed-root generation	Partial	Construction of rt_{ABC} and rt_{XYZ} is outside online proof bytes; raw-matrix encoding/root costs must be added for end-to-end deployments.
Peak memory and variance	No	Needed for a full systems artifact evaluation.

TABLE 7. MEASUREMENT COVERAGE IN THE CURRENT ARTIFACT.

Table 8 explains the proof-size convention used in the main text. The sampled matrix columns are large, but they are private Groth16 witness data; the serialized proof contains only compact handles, authentication paths, the Groth16 proof, and the batched CP-Link material.

Object	Serialized?	Where it is checked / charged
Matrix block $\mathbf{m}_{ABC,j}$	No	The sampled block is Groth16 witness data; compact commitments and Merkle paths are serialized.
Vector block $\mathbf{m}_{XYZ,j}$	No	The sampled block is Groth16 witness data and is checked against $\hat{\mathbf{x}}[j]$, $\hat{\mathbf{y}}[j]$, and $\hat{\mathbf{z}}[j]$; the SNARK-side sampled block is linked to the opened Pedersen leaf.
Merkle path	Yes	Authenticates an opened Pedersen leaf under rt_{ABC} or rt_{XYZ} ; charged to proof bytes and verify time.
CP-Link proof	Yes	One batched QA-NIZK proof τ_I links the two sampled block families to their opened Pedersen leaves.
Groth16 proof	Yes	Verifies sampled RS evaluations and fold/equality checks, while the CP-SNARK interface binds sampled block witnesses to SNARK-side commitments.

TABLE 8. PROOF-MATERIAL ACCOUNTING FOR THE CP-LINK SAMPLED-OPENING BACKEND.

Table 9 instantiates the accounting for two representative matrix sizes. The reported totals are the serialized online

proof bytes in the artifact; roots and vector commitments are public inputs, and raw sampled columns are intentionally not serialized.

Case	Opening payload	Online proof	Note
$k = 1,024$	43,904 B	44,292 B	Adds 260 B Groth16 and 128 B batched CP-Link; roots are public inputs.
$k = 4,096$	58,944 B	59,332 B	Raw sampled columns would be 50.3 MB if serialized.

TABLE 9. REPRESENTATIVE BYTE-LEVEL ACCOUNTING FROM MEOW_BENCHMARK_RESULTS_V2.CSV.

Table 10 records setup time for representative rows and separates protocol, circuit, and CP-Link setup components. The current artifact does not report serialized proving/verifying-key or CRS bytes; this is why the main text states proof size as online proof material excluding key/CRS data.

Case	Total setup	Protocol	Circuit	CP-Link	Key / CRS size
$k = 1,024$	33.16 s	0.18 s	31.83 s	1.15 s	Not reported by the current artifact.
$k = 4,096$	130.08 s	0.71 s	125.01 s	4.37 s	Not reported by the current artifact.

TABLE 10. SETUP ACCOUNTING FOR REPRESENTATIVE ROWS.

Table 11 separates the dominant deterministic $3tk$ fold terms from the remaining checked constraints. The residual includes sampled RS evaluations, equality checks, and circuit glue, so the table is a structural audit of the aggregate constraint counts rather than a per-gadget profiler.

Case	Total	$3tk$ folds	Residual	Raw cols.
$k = 1,024$	418,149	393,216	24,933	12.6 MB
$k = 4,096$	1,666,315	1,572,864	93,451	50.3 MB

TABLE 11. STRUCTURAL CONSTRAINT AND RAW-COLUMN-SIZE ACCOUNTING FOR TWO REPRESENTATIVE ROWS.

The accounting tables should be read as lower-level support for the main evaluation, not as additional benchmark claims. They justify three statements used throughout the paper: the large sampled columns remain private witness data, the serialized online proof is dominated by compact opening material rather than raw columns, and the current artifact still leaves key/CRS size and raw-matrix encoding/root construction as deployment costs outside the online proof-byte measurements.